SUMMARIZER

SQL, AND STUFF...

<Marta Lima_>
@Academia de Código
16-03-2023

HOW I FELT LAST SUMMERIZER

HOW I FEEL TODAY!

wet sidewalks are hard!

KEY CONCEPTS

JUST SOME CONCEPTS...

- **Database Design**
- Redundancy
- Design Anomalies
- Database Normalization
- Functional Dependencies
- Transitive Dependencies
- **Normal Forms**
- Stored Procedures
- Triggers
- Views
- **Concurrency**
- Transactions
- Commit
- Rollback
- Anomalies
- **ACID**
- Atomicity
- Isolation
- Durability
- **Database Indexes**

WHAT DO WE WANT FROM A DATABASE??

Good idea to write down the purpose of the database on paper — its purpose, how you expect to use it

RELATIONAL DB

DESIGNING MILESTONES

- Determine the purpose of your database
- Find and organize the information required
- Specify primary keys
- Set up the table relationships
- **Refine your design**
- **Apply the normalization rules**

REDUNDANCY

occurs when the same piece of data exists in multiple places,

INSERTION ANOMALY

occurs when a new information in the row is added, causing inconsistency.

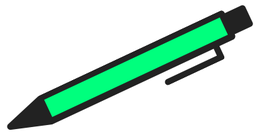
DELETION ANOMALY

you cannot delete data from the table without having to delete the entire record.

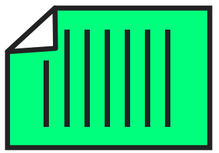
UPDATE ANOMALY :

When we update some row in the table, and it leads to inconsistency because of dupes.

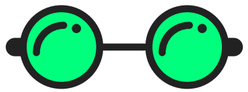
DATABASE NORMALIZATION



- Remove all the anomalies



- Distribute the data amongst different related tables



- Efficient and effective queries

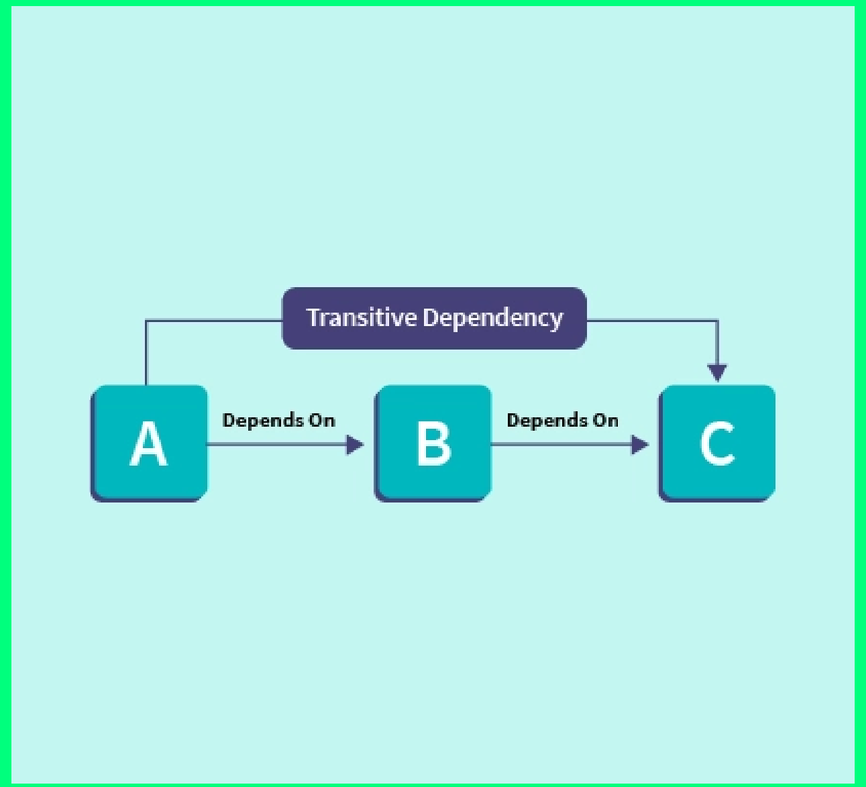
- Removes all the duplication and incorrect data issues

FUNCTIONAL DEPENDENCIES

attributes directly related

TRANSITIVE DEPENDENCIES

attributes indirectly related



HOW FAR SHOULD I NORMALIZE?

FIRST NORMAL FORM

- No repeating columns
- each column should contain single values only

SECOND NORMAL FORM

- 1NF
- all non-key attributes have to be dependent on all key attributes
- **No partial dependencies on the primary key**

THIRD NORMAL FORM

- 2NF
- every attribute has to be dependent only on key attributes
- **No transitive dependencies**

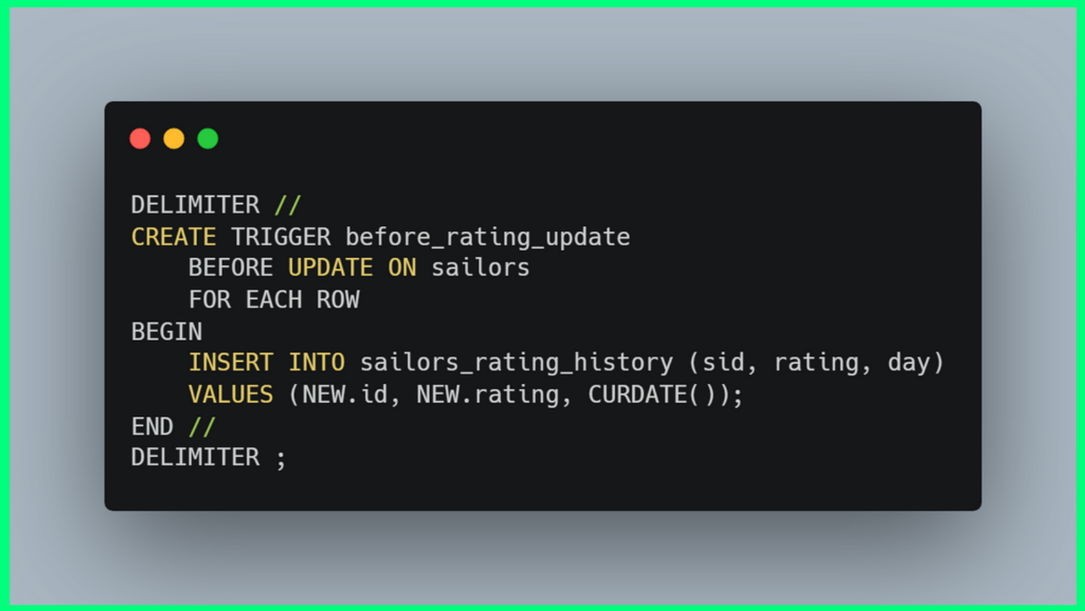
STORED PROCEDURES

```
USE boat_booking;
DELIMITER //
CREATE PROCEDURE get_reserves_by_boat_and_sailor
(IN boat VARCHAR(255), IN sailor VARCHAR(255))
BEGIN
SELECT DISTINCT * FROM reserves
WHERE reserves.bid = (
    SELECT id FROM boats WHERE boats.name = boat
) AND reserves.sid = (
    SELECT id from sailors WHERE sailors.name = sailor
);
END //
DELIMITER ;
```

```
// Call the procedure
CALL get_reserves_by_boat_and_sailor('Titanic', 'Nuno');
```

- **An application's logic is encapsulated**

- Stored procedure is a set of statement(s).
- Reusable
- similar to functions in programming.
- They accept parameters, and perform operations when we call them.



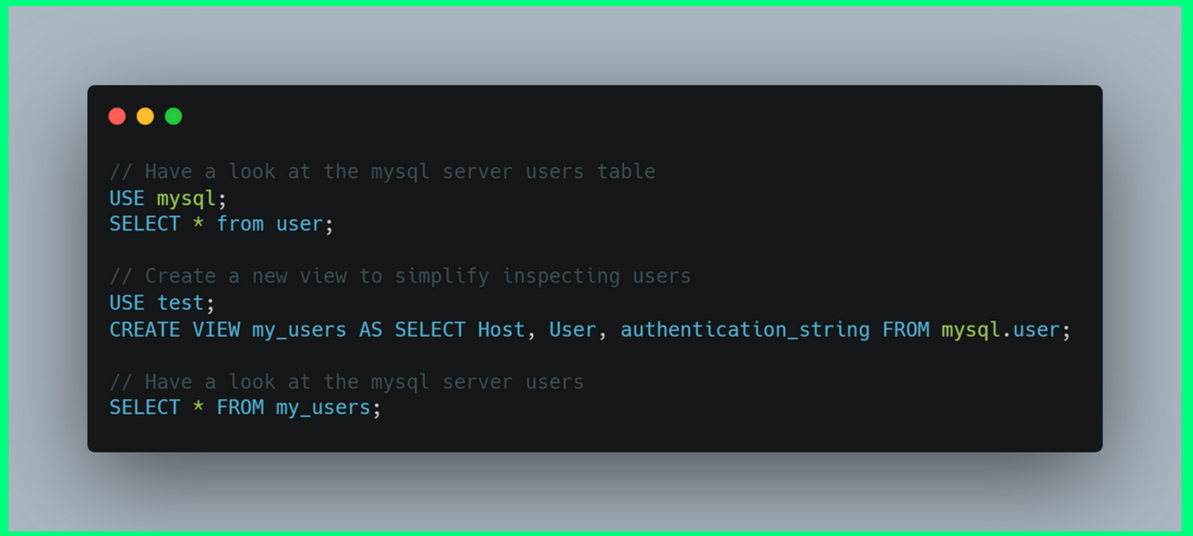
```
DELIMITER //
CREATE TRIGGER before_rating_update
    BEFORE UPDATE ON sailors
    FOR EACH ROW
BEGIN
    INSERT INTO sailors_rating_history (sid, rating, day)
    VALUES (NEW.id, NEW.rating, CURDATE());
END //
DELIMITER ;
```

TRIGGERS

are a particular type of stored procedure associated with a specific table

a named MySQL object that activates when an event occurs in a table.

VIEWS



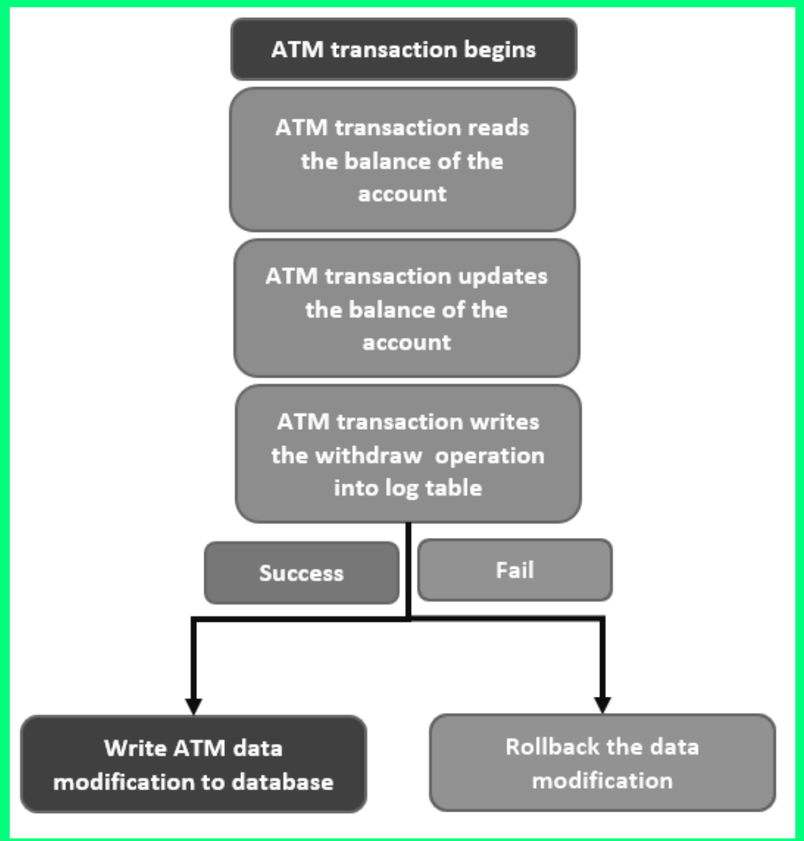
```
// Have a look at the mysql server users table
USE mysql;
SELECT * from user;

// Create a new view to simplify inspecting users
USE test;
CREATE VIEW my_users AS SELECT Host, User, authentication_string FROM mysql.user;

// Have a look at the mysql server users
SELECT * FROM my_users;
```

A view is a virtual table based on the result-set of an SQL statement.

Views are stored queries that when invoked produce a result set.



CONCURRENCY

ARE WE TALKING ABOUT JAVA????

not exactly, but we do have the same principles of parallel computing, where the integrity of the data is CENTRAL

SO WE HAVE TRANSACTIONS IN SQL

And this is how we deal with it!

TRANSACTIONS

ALL-OR-NOTHING PROPOSITION

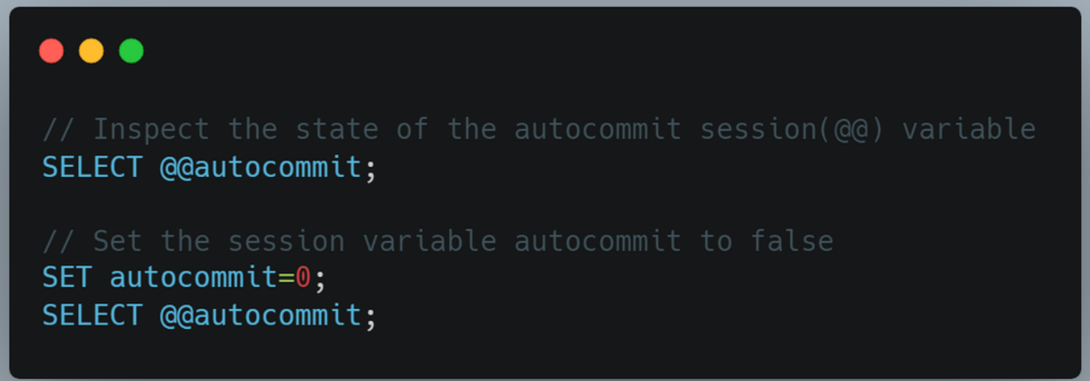
each work-unit performed in a database must either complete in its entirety or have no effect whatsoever

system must isolate each transaction from other transactions

PATTERN USED BY TRANSACTIONS

1. Begin the transaction
2. Execute a set of data manipulations and/or queries
3. If no errors occur then commit the transaction and end it
4. If errors occur then rollback the transaction and end it

AUTO COMMIT



```
// Inspect the state of the autocommit session(@@) variable
SELECT @@autocommit;

// Set the session variable autocommit to false
SET autocommit=0;
SELECT @@autocommit;
```

automatically commits all statements

SUCCESSFUL TRANSACTION

```
// Create a new table using InnoDB engine, which supports transactions
USE test;
CREATE TABLE test(num INTEGER NOT NULL) engine=InnoDB;

// Turn off auto commit
SET autocommit=0;

// Execute transaction, perform queries from other mysql connections
START TRANSACTION;
INSERT INTO test VALUES (1, 2, 3);
INSERT INTO test VALUES (4, 5);
COMMIT;
```

automatically commits all statements

TRANSACTION WITH ERRORS

```
// Truncate table test
TRUNCATE test;

// Execute transaction, perform queries from other mysql connections
START TRANSACTION;
INSERT INTO test VALUES (1, 2, 3);
INSERT INTO test VALUES (4, 5);

// Some error occurs.... let's rollback

ROLLBACK;
```

ACID Properties of Transactions

- **ATOMICITY**

Either they execute in full or they don't execute at all.

- **CONSISTENCY**

any transaction will bring the database from a valid state to another valid state.

- **ISOLATION**

guarantees that concurrent execution of transactions results in a system state that would be obtained if transactions were executed serially

- **DURABILITY**

Once a transaction is successfully finished, all changes are immutable, surviving any kind of fault.

DATABASE INDEXES

DATA ACCESS MORE EFFICIENT

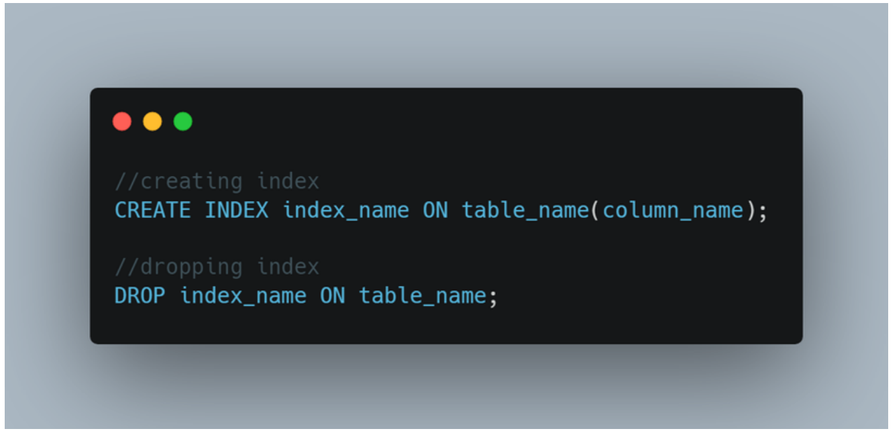
Most used queries.
Prepare the RDBMS to better perform such queries

USING INDEXES

speed up data access,

find all rows that match a given column in a query.

CREATING INDEXES



FIM!